Neutron-2 Software Design Document

University of Hawai‘i at Mānoa

Hawai‘i Space Flight Laboratory

University Nanosatellite Program

Software Lead

Aris Carlos
University of Hawai‘i at Mānoa
Email: atcarlos@hsfl.hawaii.edu

Project Manager

Piper Kline
University of Hawai‘i at Mānoa
1680 East-West Road, POST 529
Honolulu, HI 96822
Email: pkline@hsfl.hawaii.edu

Principal Investigator

Dr. Miguel Nunes
University of Hawai‘i at Mānoa
1680 East-West Road, POST 522
Honolulu, HI 96822
Phone: 808 956 0441
Email: miguel.nunes@hsfl.hawaii.edu

Revision/Date

Rev 4.0 / 2026-09-17

Revision History

Revision / date	Summary
1.0 / April 2025	Initial software design for the System Requirements Review.
2.0 / 29 September 2025	Preliminary Design Review deliverable: F′ selection, project organization, configuration management, preliminary architecture and FDIR.
3.0 / 2026	PDR feedback incorporated; team working revision. The 44 review comments and their dispositions are filed with this document.
4.0 / 17 September 2026	Critical Design Review deliverable. Requirements allocated to RVM v4.0 sheet 6.7 FSW; seven-mode design conformed to MOP v2.0; FDIR tiers and no-ECC memory integrity; command authentication and data protection; iOBC platform status and resource budget; verification gates; open decisions.

Table of Contents

1. Introduction 4
    1.1. Purpose 4
    1.2. Scope 4
    1.3. Overview 5
    1.4. Acronyms and terms 5
    1.5. Tables and figures 6
2. Design Basis 6
    2.1. Ownership 7
    2.2. Baseline decisions 7
3. Software Architecture 8
    3.1. Layers 9
    3.2. Command authority and framework services 10
    3.3. Why F′ 10
    3.4. Component responsibilities 11
    3.5. Component contracts 12
4. Interfaces and Execution 13
    4.1. Driver allocation 13
    4.2. Common contracts 13
    4.3. Scheduling and resource isolation 13
    4.4. F′ execution 14
5. Operational Modes and Sequences 14
    5.1. Nominal science sequence 16
    5.2. Interrupted collection and restart 17
6. Science Data and Ground Operations 19
    6.1. Product and metadata model 19
    6.2. Storage and retention 19
    6.3. Data ownership and persistence 20
    6.4. Ground system 21
    6.5. Ground tools and workflow 21
7. Fault Detection, Isolation, and Recovery (FDIR) 22
    7.1. Safe entry and authority 22
    7.2. Safe exit 22
    7.3. Memory, state integrity, and radiation 25
8. Command Authentication and Data Protection 26

8.1. Requirement basis 4
8.2. Threat scope 4
8.3. Protection policy by data class 4
8.4. Design in F′ v4.3.0 4
8.5. Three constraints 4
8.6. Frame profile 4
8.7. Key handling and replay protection 4
8.8. Requirement trace and non-goals 4
9. Flight Platform and Configuration 4
9.1. iOBC feasibility milestone 4
9.2. Resource and release control 4
9.3. Boot and update boundary 4
10. Development and Verification 4
10.1. Testbeds and evidence 4
10.2. Requirement allocation 4
11. Open Decisions 4
12. Student Development and Team Workflow 4
12.1. Weekly work and definition of done 4
12.2. Branches and review authority 4
12.3. Coding and verification practices 4
12.4. Onboarding 4
Appendix A. How Commands and Telemetry Move Through F′ 4

1. Introduction

1.1. Purpose

This document is the software design for the Neutron-2 flight software. It states what the software does, which component owns each responsibility, how the design meets the flight software requirements, and what evidence is required before each hardware stage. It is written for students joining the software team.

1.2. Scope

In scope: onboard command and telemetry handling, mission modes, payload sequencing, science storage and downlink, subsystem coordination, fault detection and recovery, command authentication, and the flight-to-ground interface. Out of scope: payload acquisition firmware, vendor ADCS control algorithms, electrical protection circuits, RF performance, and ground

operating procedures. Those belong to their subsystems; this document defines only the software interface to them, completed through the ICD.

1.3. Overview

The flight software is C++ on the F Prime (F′) framework. Mission logic is shared between the Artemis FlatSat engineering model and the flight build; only the hardware adapters change. The flight computer is the ISIS iOBC running FreeRTOS, the only operating system the vendor documents for the board [S4]. F′ does not list the iOBC as a supported platform, so the port is work for the team with three open items: the application code budget against 1 MB of NOR flash, the operating-system abstraction port, and the scheduling model (Section 9.1, OD-01).

New students should read Section 3 for the architecture, Section 5 for the mission sequence, and Section 12 for the contribution workflow before taking a task.

1.4. Acronyms and terms

ADCS — Attitude determination and control system.

APID — Application process identifier; the CCSDS Space Packet field that identifies a data stream.

BSP — Board support package.

CCSDS — Consultative Committee for Space Data Systems.

CTL — Command and Telemetry Dictionary.

ECC — Error-correcting code memory.

EGSE — Electrical ground support equipment.

EPS — Electrical power system.

FDIR — Fault detection, isolation, and recovery.

FPP — The modeling language for F′ components, ports, types, and topologies.

FRAM — Ferroelectric random-access memory; the iOBC’s byte-writable non-volatile store.

FSW — Flight software.

GDS — Ground data system.

HIL — Hardware-in-the-loop testing.

ICD — Interface control document.

iOBC — ISISPACE onboard computer.

MAC — Message authentication code.

MDD — Mission Design Document.

MOP — Mission Operations Plan.

OSAL — Operating-system abstraction layer.

PDU — Power distribution unit.

RTOS — Real-time operating system.

RVM — Requirements Verification Matrix.

SA — Security association; in SDLS, an indexed protection policy applied to a transfer frame.

SDLS — Space Data Link Security; the CCSDS frame-level protection layer.

SOH — State of health.

TBD / TBC — To be determined / to be confirmed.

VC — Virtual channel; a CCSDS transfer-frame subdivision of a physical link.

1.5. Tables and figures

Tables 1 to 14 cover sources, decisions, components, drivers, modes, data ownership, fault responses, fault tiers, memory integrity, protection policy, verification gates, requirement allocation, open decisions, and the resource budget. Figures 1 to 5 show system context, software layers, mission modes, nominal science, and recovery. Appendix A maps the F′ CCSDS and SDLS components used in Section 8.

2. Design Basis

The RVM defines required behavior. The MOP defines operating policy, including mode definitions and permitted transitions. The CTL defines the command and telemetry service set. The ICD defines external interfaces. This document is the design that implements them. Where they disagree, Table 13 records the conflict and the owner who decides it.

Table 1. Sources

ID	Source and use
S2	RVM workbook, filed as v3.10, and the current FSW requirements allocation, sheet 6.7 FSW, FSW-001 to FSW-015. The FSW allocation controls current requirement wording, source references, verification methods, and status.
S4	ISIS On-Board Computer Datasheet, ISIS.ISIS-OBC.DS.1.1, Issue 1.3, 21 December 2018. The controlled iOBC specification. The undated product brochure in the same folder disagrees on SDRAM (64 MB versus 32 MB) and operating systems; this document follows the datasheet. Both describe advertised capability, not the delivered unit.
S5	MDD v5.0; MOP v2.0 §§7–8; ICD v2.0. Mission and interface authority.

ID	Source and use
S6	fprime-artemis-cubesat, branch neutron_2, commit abc8bcd5e354f1f87c2d8e52b0499a09c992e110, F′ submodule v4.2.1. The existing FlatSat prototype.
S8	F′ v4.3.0, commit 7d8f579f159d2f7c2d4984d92828575e37f87fa6. Framework architecture, CCSDS and SDLS subtopologies, supported-platform status, porting guidance.
S9	NASA JSC radiation reference and JPL ASIC radiation-effects guidance. Used only for the hardware/software fault boundary in Section 7.3.
S11	N2.PM.Command & Telemetry Dictionary v1.0 (CTL). Defines FSW-014 and FSW-015.
S12	ISIS-OBC Quick Start Guide v2.2. Boot chain, NOR bootloader region, image programming, SDK and toolchain.
S13	Internal iOBC studies in the fprime-artemis-cubesat repository (docs/archive): F′ version memo and NOR fit analysis. Paper analysis on an ARMv6 Linux proxy; no iOBC bring-up has been performed.
S14	N2.PM.System Budgets v2.1. Mass storage and data budgets.

Status labels used in this document: selected means a team design choice; provisional means an assumption awaiting owner review; existing prototype means capability present in S6; planned means future implementation.

2.1. Ownership

The software lead owns architecture, technical review, integration readiness, and mentoring. Each student owns an assigned issue, including implementation, tests, and handoff notes. Subsystem leads own device behavior and ICD inputs. Systems engineering owns cross-subsystem allocations. Mission operations owns operating procedures and recovery policy. The project manager coordinates milestones. Individual names live in the team roster, not here.

At semester handoff, retain a reproducible demo, a tested release tag, the issue backlog, interface decisions, as-run test records, and a setup guide. Pair each new contributor with an experienced reviewer so no component depends on one graduating student.

2.2. Baseline decisions

Table 2. Baseline decisions and status

Decision	Status
C++ and F′, shared mission logic, target-specific adapters	Selected for FlatSat and flight.

Decision	Status
iOBC flight target, FreeRTOS	Selected. The iOBC is not an F′-supported platform, so the milestone is a port: code budget against 1 MB NOR, OSAL port, and scheduling model are open (Section 9.1, OD-01).
Safe exit and interrupted-collection restart	Safe exit is ground-commanded (MOP v2.0 §8.8, FSW-001). MissionApp enforces exit guards on the command. Restart requires a valid science window and all admission checks (OD-03/04).
Retain science until confirmed delivery; onboard reduction for the UHF backup path	Selected. Raw and reduced-volume products are separate classes (Section 6). Only ground-confirmed products are reclaimed (OD-02/05).
Ground and student workflow	Yamcs for flight; F′ GDS for FlatSat. Weekly Scrum; student PRs to develop; leads manage release and tags.
CCSDS conformance; PUS service concepts borrowed, PUS conformance not claimed	Selected. FSW-009 requires CCSDS Space Packets; F′ v4.3.0 implements CCSDS 133.0-B-2; the RVM proposes 133.0-B-2 EC2 §4.1, pending edition/profile confirmation. FDIR does not depend on PUS services.
Command authentication and uplink protection	Selected: the F′ v4.3.0 SDLS integration interfaces connected to an N2 cryptographic component, pre-loaded symmetric keys, no in-flight rekey. Onboard allocation, cipher and MAC selection, and the packet profile are OD-13.
Single satellite is the primary case	Selected. The spacecraft identifier stays in the product schema as a design choice and in the command MAC, so a second unit needs no redesign. Two-spacecraft assumptions in the MDD and MOP are OD-10.

3. Software Architecture

The iOBC runs the team-owned C++/F′ deployment. The payload computer and the vendor ADCS algorithms are outside that boundary. The Artemis Pi/Teensy arrangement is the FlatSat transport. Figure 1 separates the team’s software from external devices and ground tools. [S5, S6]

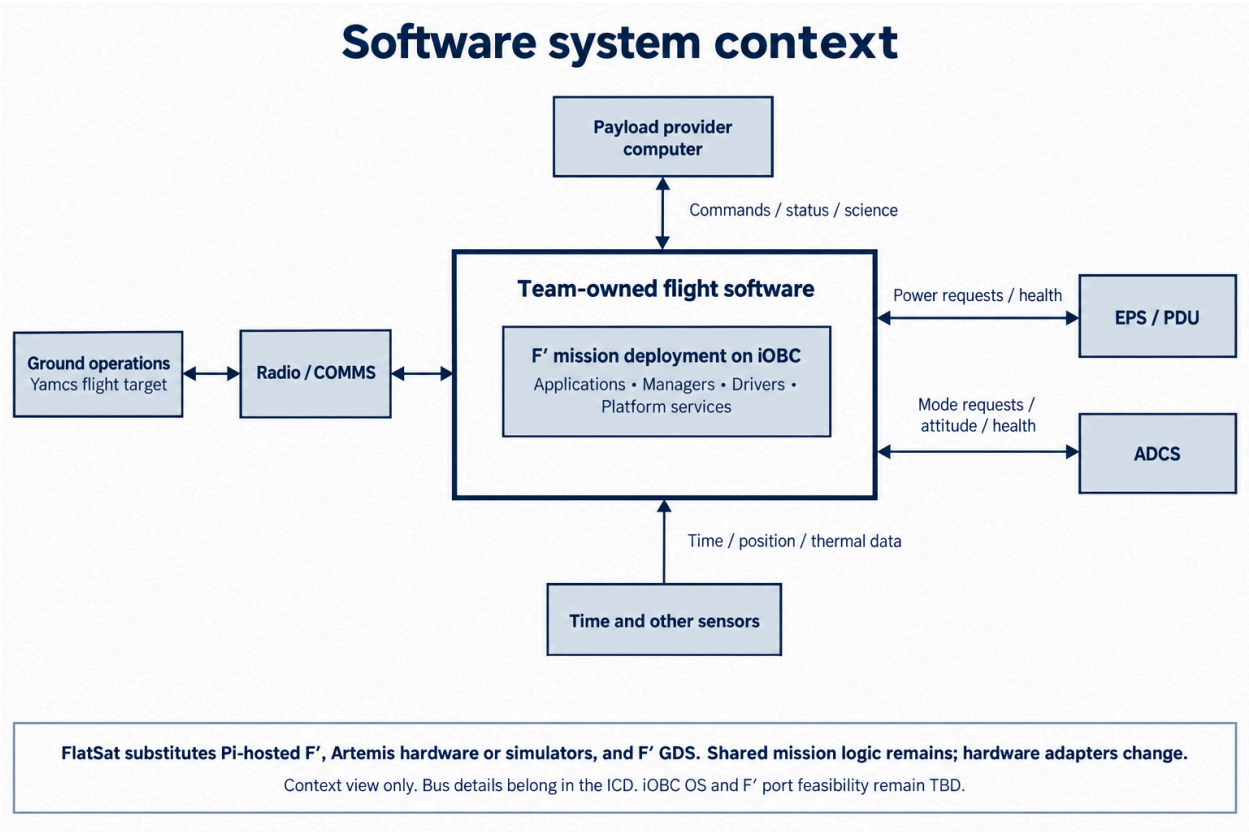


Figure 1. Software system context

The context boundary shows responsibility, not electrical layout. Flight bus selections and message formats are in the ICD.

3.1. Layers

Applications own mission sequences and policy. Managers expose typed, hardware-independent subsystem operations and report achieved state, validity, and errors. Drivers adapt those contracts to the selected hardware or simulator. Vendor SDK calls stay below the manager contract; mission applications never call board, OS, or vendor APIs directly. Figure 2 shows the layers; Table 4 lists the hardware substitutions. [S6, S8]

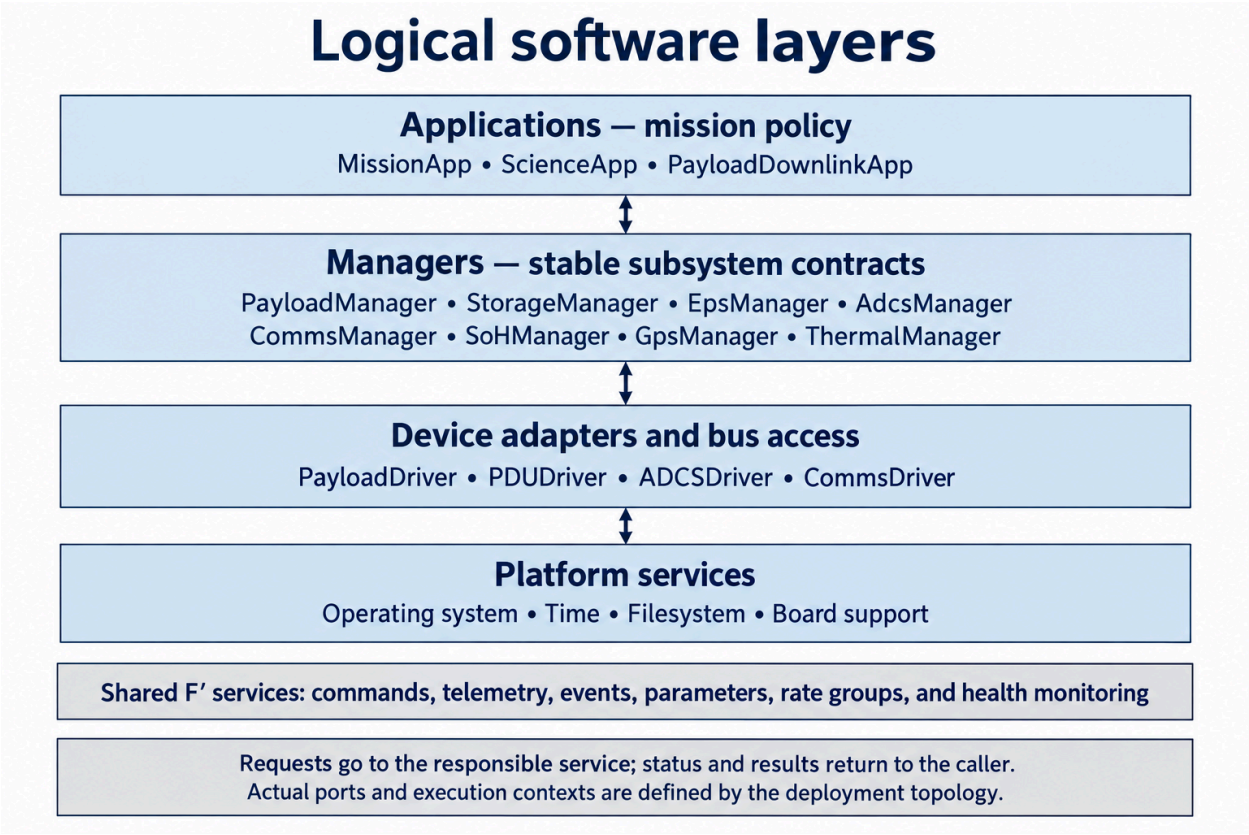


Figure 2. Logical software layers

Each deployment is a separately built executable. Host simulation runs mission logic without hardware. FlatSat runs on the Pi-hosted F′ with Artemis bridge hardware. Flight targets the iOBC after its feasibility gate. Subtopologies group services, mission logic, managers, and platform adapters inside one deployment; they are not separate processors.

3.2. Command authority and framework services

MissionApp is the only component that changes spacecraft mode. ScienceApp and PayloadDownlinkApp request or execute authorized activities. Subsystem managers never change mode. Hardware protection may act on its own and then report its state to the software. Ground requests pass through the command service and the mode and interlock checks.

The design reuses F′ command dispatch, events, telemetry, parameters, rate groups, health monitoring, and bounded buffer and file services. Component execution types, queue sizes, task priorities, and the inclusion of sequencer and file-transfer services are chosen and measured per deployment. Framework health monitoring does not replace the mission fault policy or the hardware watchdog.

3.3. Why F′

F′ provides typed component interfaces, generated command and telemetry definitions, test harnesses, and a working FlatSat starting point. Its costs are learning FPP and C++, managing asynchronous execution, and building the iOBC port. If the iOBC feasibility milestone fails, the team documents a platform trade and decides before changing the baseline. [S6, S8]

3.4. Component responsibilities

Table 3 assigns responsibilities. This document uses Application–Manager–Driver names; the neutron_2 branch uses Manager–Service–Adapter for the same roles (MissionApp is MissionManager, ScienceApp is ScienceManager, PayloadDownlinkApp is PayloadDownlinkManager; subsystem managers map to Service components). The layering has no requirement parent (OD-15). [S6]

Table 3. Component responsibilities

Component	Responsibility
MissionApp	Owns the seven modes, command admission, scheduled activity, transition guards, Safe entry, exit-guard enforcement on a commanded Safe exit, the ground-contact timer, and the commissioning flags evaluated on every mode entry (MOP v2.0 §8).
ScienceApp	Runs collection and payload-to-OBC transfer. Coordinates PayloadManager, context sources, and StorageManager; reports completion, partial results, and failure to MissionApp.
PayloadDownlinkApp	Selects stored products, requests transfer through CommsManager, tracks progress and ground receipt, updates delivery state. Transmission alone never authorizes deletion.
PayloadManager	Exposes standby, configuration, start/abort, status, and product-transfer operations. Enforces legal payload states; delegates protocol details to PayloadDriver.
StorageManager	Owns storage admission, the product catalog, integrity and completion state, partial products, and retention. Reclaims only ground-confirmed products.
EpsManager	Exposes power-state requests, output status, and EPS health and fault indications through the Artemis PDU or flight EPS adapter. Electrical protection stays in hardware.
AdcsManager	Requests vendor attitude modes and consumes attitude and health with validity. Monitors reported angular rates against the safety range and commands no control while it is exceeded (FSW-011); reports the inhibit to MissionApp and SoHManager. The range and hysteresis are TBD, owned by ADCS and systems engineering.

Component	Responsibility
CommsManager	Exposes link readiness, radio-state requests, prioritized transport, and errors through the radio adapter. Keeps the command and SOH service available during science transfer.
GpsManager, ThermalManager	Provides position, time, and thermal observations with age, validity, and fault indications. Source selection and thermal-control authority are open.
SoHManager	Aggregates subsystem health and validity for telemetry and fault evaluation.
Supporting services	Scheduling, deployment inhibitors, time correlation, and fault evaluation extend existing components or use small helpers. Boundaries are TBD.

3.5. Component contracts

Operation names below describe intent. Typed ports, command IDs, queue sizes, and execution types belong in the FPP model. The remaining managers follow the same pattern using Table 3 and Section 4.2.

MissionApp
Inputs: Ground commands, schedule triggers, subsystem health, activity results.

Outputs: Accepted or rejected requests, mode changes, authorized science or recovery actions.

State and failure: Owns mode, schedules, and recovery policy. Rejects failed guards. After reset, establishes the inhibited startup state and revalidates persisted activity before use.

ScienceApp
Inputs: An authorized collection request, payload status, storage results.

Outputs: Payload acquisition, Standby, and transfer requests; product writes; completion or failure reports.

State and failure: Owns the active sequence and attempt identity. On timeout or cancel, preserves the partial attempt; a restart gets a new attempt ID.

PayloadManager
Inputs: Standby, configuration, start/abort, product-transfer requests; driver status.

Outputs: Adapter requests; achieved-state, progress, completion, or error reports.

State and failure: Owns payload operation state. A missing response is unknown state. Reconciles with the device after reset before allowing acquisition.

StorageManager
Inputs: Capacity reservations, bounded product writes, finalization requests, delivery updates.

Outputs: Admission decisions, product handles and status, available capacity, integrity errors.

State and failure: Owns files and catalog. Reconciles catalog against media after reset; marks incomplete writes partial. Failed admission blocks dependent collection.

PayloadDownlinkApp
Inputs: An authorized downlink request, product metadata, transport results, ground receipt messages.

Outputs: Chunk transfers through CommsManager, progress reports, delivery updates to StorageManager.

State and failure: Owns the downlink session. Preserves product identity and progress on interruption. Ground integrity confirmation, not transmission, enables deletion eligibility.

4. Interfaces and Execution

4.1. Driver allocation

Table 4 allocates prototype and flight adapters. Addresses, pins, framing, rates, and timeouts are in the ICD and vendor documents, not here. Driver names are design labels until implemented and tested.

Table 4. Driver allocations

Contract	Artemis FlatSat	Flight target and closure
PayloadDriver	Neutron simulator; next increment is the real payload adapter.	Payload RS-422/UART path (OBC-006). Freeze product format, states, completion, timeout, and abort semantics.
PDUDriver / EPS adapter	PDU v2 adapter exists; physical PDU HIL and EPS-powered bring-up are unverified.	Flight EPS adapter, candidate I²C. Do not reuse the Artemis wire protocol as the flight contract.
ADCSDriver	Simulation or prototype source.	CubeADCS adapter, I²C candidate. Confirm interface and state/validity mapping.
CommsDriver	Teensy/RFM23BP bridge and ground receiver.	SatNOGS COMMS candidate over SPI. Confirm framing, link profile, packet limits, Yamcs adapter.
GPSDriver / ThermalDriver	Simulated or prototype sources.	Flight receiver and thermal sources TBD. Confirm timing source and ownership.
StorageDriver / platform	Pi/Linux storage.	iOBC SDK, FAT32 filesystem, selected media. OSAL, BSP, boot, watchdog, and storage behavior need target tests.

4.2. Common contracts

Every asynchronous request carries a correlation identifier and has an explicit completion or failure result. Subsystem reports distinguish requested from achieved state and carry observation time, source validity, and age; a stale reading is reported as stale, not as a value. Commands validate arguments, mode permission, readiness, and inhibits before execution; acceptance and completion are separately observable.

Product operations identify product and attempt, chunk size, offset, completion state, and integrity result. Storage operations define buffer ownership and lifetime. Adapters serialize transactions where required and enforce bounded timeouts and retries. A missing response is an unknown state, never success. Final FPP types and limits are ICD and dictionary work.

4.3. Scheduling and resource isolation

Long payload, storage, and radio transactions run outside time-critical polling and health paths. Bulk data moves in bounded chunks with backpressure; unavailable buffers delay or reject work rather than allocating. Queue overflow, stale telemetry, storage exhaustion, and timeouts produce observable results. Priorities and rates must keep command handling, health reporting, and fault response alive during collection or downlink.

Bulk science shares the framer and the SDLS stage with telemetry and events. The isolation available is queue depth, queue priority, and buffer binning, not path separation (Section 8.5). Throughput under simultaneous science downlink and telemetry is a Section 10 gate measurement. [S8]

Time-tagged and geolocation-tagged schedules persist with identifier, trigger, validity window, duration, execution state, and restart policy. After reboot or recovery, triggers are revalidated against valid time and location; a trigger with an invalid source is held. Persistence format and trigger tolerance are TBD.

4.4. F′ execution

A component owns behavior; a typed port is its interface; a topology instantiates and connects components. Passive components have no thread or queue. Active components have both. Queued components have a queue that another context dispatches to. Synchronous calls run on the caller’s thread; guarded calls also serialize; asynchronous calls queue at the receiver. These are chosen deliberately, not by naming convention. [S8]

Rate-group callbacks stay short and bounded. Slow payload, file, and radio work runs in a worker or an incremental state machine with explicit completion, timeout, and backpressure. Buffers are returned on success, failure, cancellation, and reset. F′ Health pings prove software responsiveness only. These patterns assume a pre-emptive OS, which the iOBC’s documented FreeRTOS configuration is not; Section 9.1 carries that item, and until it closes, every long synchronous call is treated as blocking the whole deployment. [S4, S8]

5. Operational Modes and Sequences

Table 5 defines the seven flight modes from MOP v2.0 §8. FSW-001 requires the software to perform each permitted mode change defined there when its manual, queued, or automatic trigger occurs, and its entry conditions are met. FSW-013 requires booting to a safe mode with minimal subsystem activation after any reset or power cycle. The neutron_2 code implements five demo states (BASE, COLLECTION_PENDING, COLLECTING, SCIENCE_READY, DOWNLINKING); the seven-mode model is a planned extension. A transition completes only after the required achieved states are confirmed. [S2, S5, S6]

A guard is a condition that must pass before a transition or operation is allowed; a failed guard rejects and reports the transition. Figure 3 shows the principal transitions. Implementation tests must cover every enabled transition and its guards.

MOP v2.0 §8 also requires commissioning flags. On every mode entry, MissionApp evaluates the persisted flags and assigns each subsystem the operational state its commissioning status allows; for example, ADCS in Base is Y-Thomson until the three-axis flag is set, then nadir pointing. Flags change only by ground command and are recorded in the critical parameter store (Table 6). A missing or invalid flag selects the least-capable state.

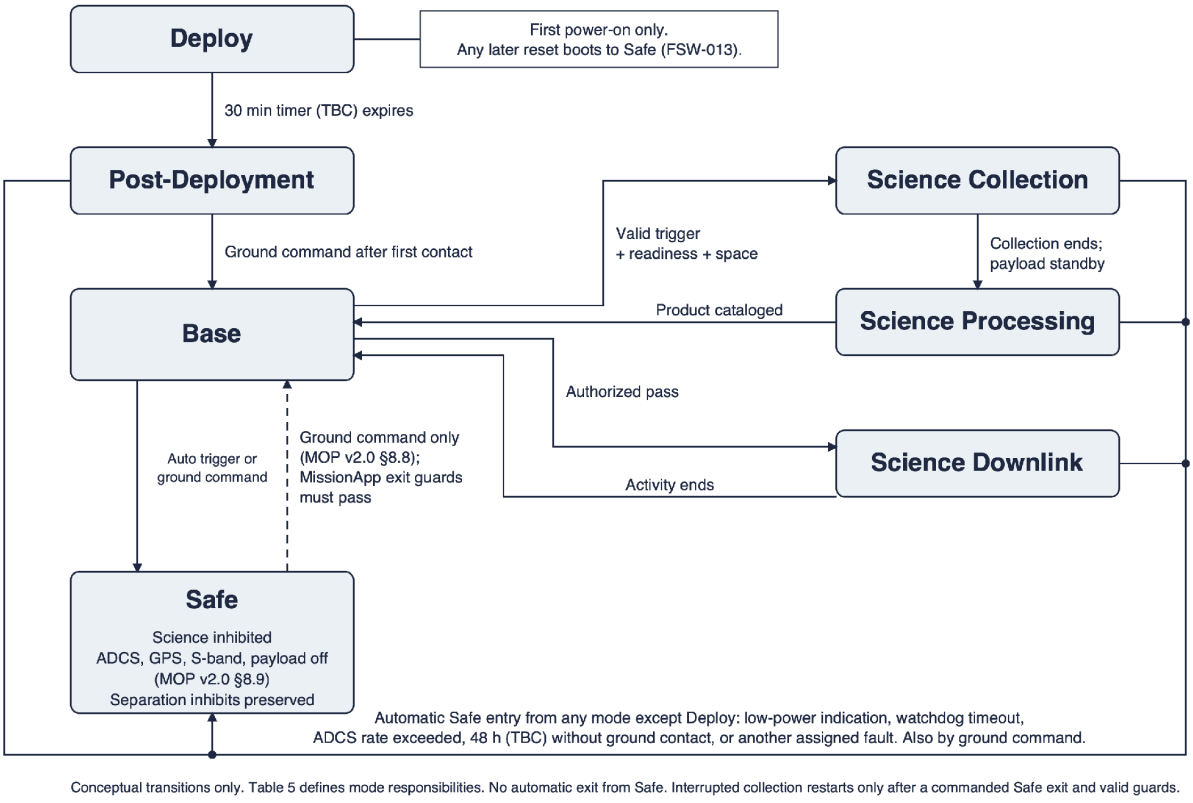


Figure 3. Target mission mode transitions

Table 5. Operational modes

Mode	Purpose and progression
Deploy	Proposed for first power-on only, pending reconciliation with FSW-013 (Section 9.3, OD-07). Separation-related RF, deployable, and attitude-command inhibits apply. The iOBC runs a 30-minute (TBC) timer, then transitions automatically to Post-Deployment.
Post-Deployment	Antenna deployment, auto-detumble with the FSW-011 rate monitor active, initial contact. Ground commands the transition to Base.
Base	Health, scheduling, storage, and communications readiness. Admits valid collection or downlink requests. Distinct from Safe: the MOP calls Base “a safe mode” informally; FSW-013’s safe mode is Safe Mode below.
Science Collection	Commands the payload from Standby into acquisition for the authorized duration. Payload owns acquisition and buffering; FSW collects health and context. On completion, the payload returns to Standby and the mode changes to Science Processing.
Science Processing	Transfers the completed product to OBC storage, attaches metadata, validates transfer. Generates the FSW-012 reduced-volume product for science selected for the UHF backup path. Returns to Base when products are cataloged, or the failure response completes.
Science Downlink	Transfers selected products and telemetry through COMMS. Preserves product state across interruption; returns to Base when the activity ends.
Safe	Science inhibited; ADCS, GPS, S-band, and payload off; EPS and OBC minimal (MOP v2.0 §8.9, TBC). Health reporting continues. Entered automatically on a low-power indication, watchdog timeout, ADCS rate exceedance, 48 hours (TBC) without ground contact, or another assigned fault indication (FSW-007, MOP v2.0 §8.8), or by ground command. Exit only by ground command, guarded by Section 7.2.

5.1. Nominal science sequence

Ground enables a valid activity. MissionApp checks mode, trigger, payload readiness, storage admission, and active faults. ScienceApp commands collection through PayloadManager. When acquisition ends, the payload returns to Standby and transfers the product to OBC storage. StorageManager marks the product complete only after the interface completion and integrity checks pass. A later contact transfers the product and its context; ground receipt is recorded separately from transmission. Figure 4 shows the accepted path.

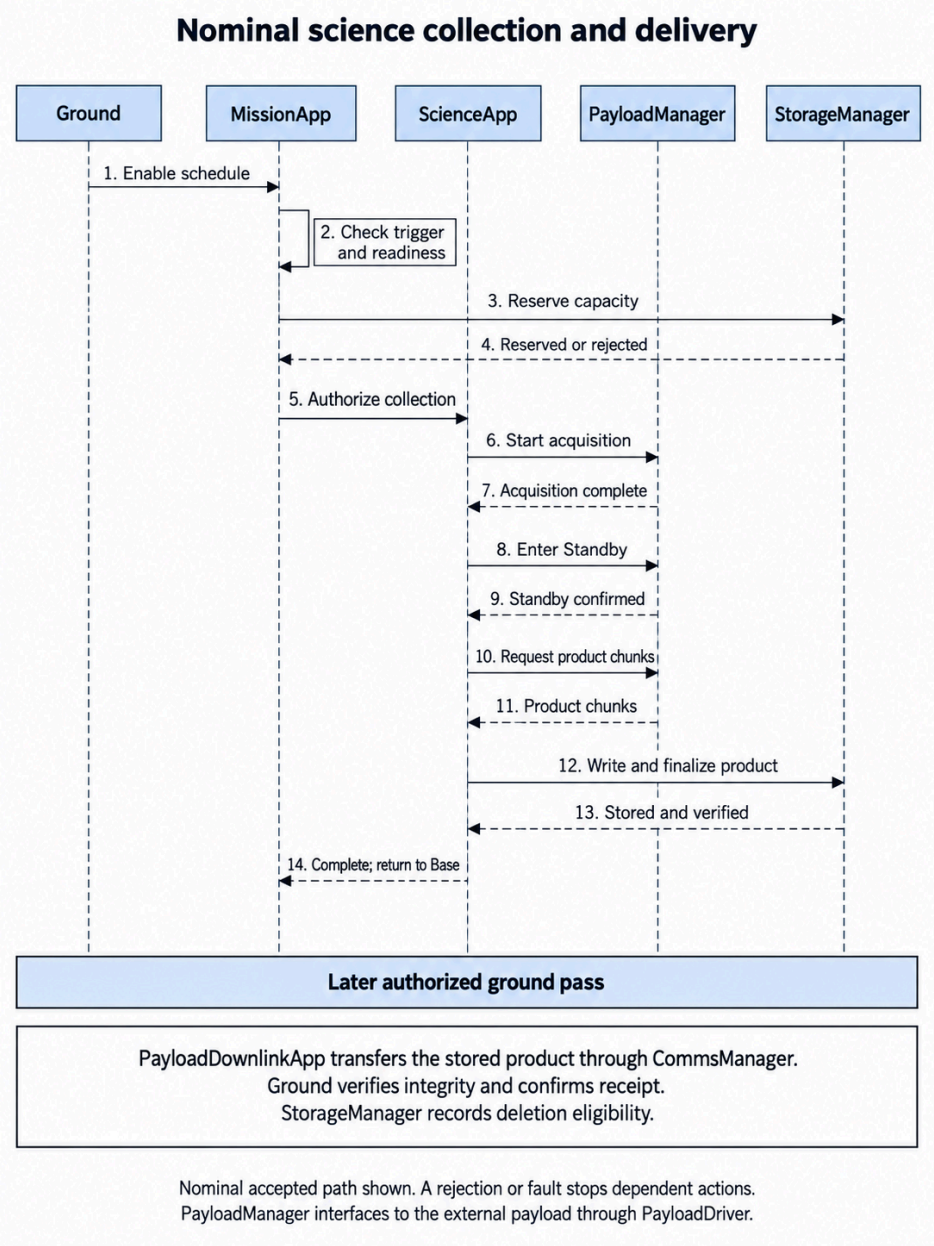


Figure 4. Nominal science collection and delivery

5.2. Interrupted collection and restart

After a ground-commanded Safe exit, MissionApp may restart an interrupted collection if its science window is still valid and all admission checks pass. The prior attempt is kept as partial, and the restart gets a new attempt identifier. Working assumption: enough time must remain to complete the requested duration; otherwise the activity is skipped and logged. Systems engineering and the payload and operations owners decide whether shortened collections are allowed (OD-04).

Payload reset, uncertain device state, stale trigger inputs, insufficient storage, or a critical fault blocks restart. Repeat attempts are bounded. Figure 5 shows the sequence; its subsystem lane groups the payload, storage, EPS, and ADCS managers. Protective actions do not wait for science cleanup; missing acknowledgments remain faults.

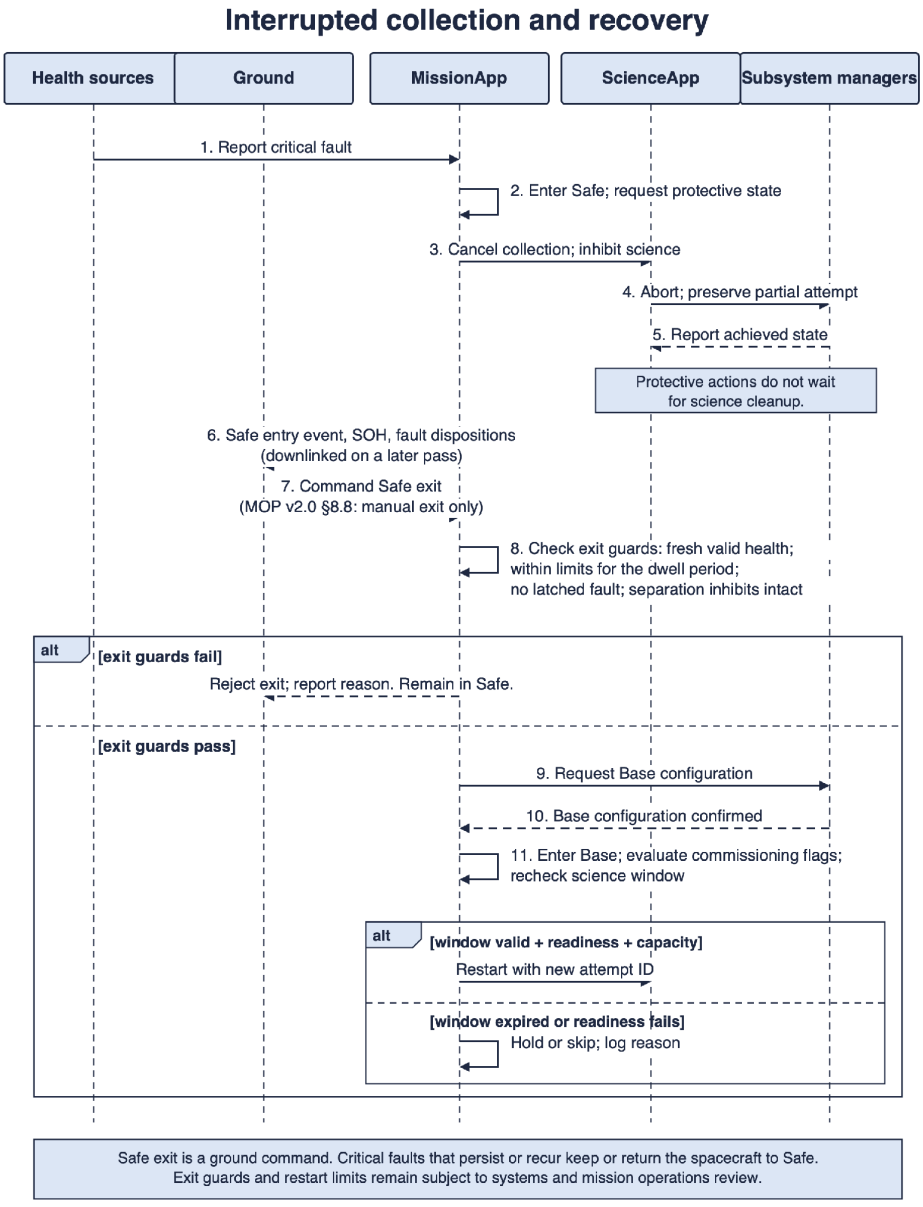


Figure 5. Interrupted collection and recovery

6. Science Data and Ground Operations

6.1. Product and metadata model

The payload acquires internally and supplies CSV-like science products. The OBC transfers, stores, tags, and downlinks them, and generates a reduced-volume version of selected science for the UHF backup path (FSW-012). CSV encoding is a working assumption, not an approved payload contract. Format, maximum product size, cadence, and retention are confirmed with systems engineering and the payload team (OD-02). [S2, S5]

Two product classes exist. A raw product preserves the payload bytes unchanged. A reduced-volume product is derived from one or more raw products by compression or summarization; it is a separate catalog entry that names its source product and attempt, carries its own integrity value, and never replaces its source. Ground confirmation of a reduced product does not make the raw product deletion-eligible. The reduction algorithm, its CPU and memory cost on the 400 MHz ARM9, and the integrity meaning of a derived product are OD-02.

Each product carries the spacecraft identifier, product and attempt identifiers, collection start and end time, payload configuration, completion status, integrity information, and position, altitude, attitude, and magnetic-field context with validity and source timing. FSW-005 requires time, latitude, longitude, altitude, attitude, and magnetic-field direction and magnitude for the measurement interval, so the magnetic-field fields are mandatory. Invalid context is flagged, never replaced with a fabricated value. The data dictionary allocates per-record versus per-product context.

Time. OBC-010 requires onboard time synchronized to UTC within ±0.5 s, and FSW-005 requires time to be recorded for the science measurement interval, so clock discipline is software work. The iOBC RTC is the running clock. A valid GPS time fix corrects it; it logs each correction and its age. Every timestamp carries its clock source and estimated error. After a reset, the clock is unsynchronized until the next valid fix or a ground time update; records taken in that state are flagged, not discarded, and ground correlates them from the logged corrections. Drift measurement, the accuracy budget, and the ground time-update command are OD-08. [S2]

6.2. Storage and retention

The catalog distinguishes raw and reduced products and, for each, the partial, complete, selected, transmitted, ground-confirmed, and deletion-eligible states. Receipt acknowledgment identifies the product and integrity value so confirmation cannot land on the wrong attempt or class. Missing or ambiguous confirmation leaves the product retained.

FSW-006 requires storing each state-of-health record received from a powered subsystem during the previous 48 hours and making it available to COMMS for downlink. That window is the floor on the engineering-log allocation and the fault-evidence budget Section 7 spends. System Budgets v2.1 sizes it at about 978 kB; OBC-002 carries the hardware side. [S2, S14]

When space is needed, it deletes the oldest ground-confirmed science products first. Unconfirmed science is never deleted automatically. If eligible products cannot free enough space, it rejects or holds the next collection and reports storage pressure. Engineering and event logs have a separate bounded allocation. Reserve size, ordering key, protected products, and log retention are TBD (OD-05).

Capacity is reserved before collection for the maximum bounded product, partial-attempt overhead, metadata, and the derived product. Products stream in chunks; the reduction stage reads its source in the same bounded chunks. After reset, catalog entries are reconciled against stored files; incomplete writes are marked partial, and a derived product whose source is partial is itself partial. Atomic-write and media-recovery mechanisms depend on the iOBC filesystem.

6.3. Data ownership and persistence

Table 6 states the data lifecycle. Persistent means required to survive reset. Transient data is rebuilt. A saved value never substitutes for a fresh observation when a safety or readiness check needs one. The last four rows are served by the critical parameter store of Section 7.3.

Table 6. Data ownership and persistence

Data and owner	Persistence and reset behavior	Validity or confirmation
Mode and recovery state (MissionApp)	Initialize a known inhibited state after reset. Retain only approved fault latches; a saved mode is history, not authority to resume.	Fresh health, clear dwell, and achieved Base state required.
Schedules and attempts (MissionApp, ScienceApp)	Persist window, policy, and attempt identity and status. Revalidate triggers after reset; interrupted attempts stay partial.	New attempt admitted only when window and guards pass.
Science files and catalog (StorageManager)	Persist product and attempt IDs, metadata, integrity, and partial/complete/corrupt state. Reconcile against media after reset.	Complete only after transfer and storage validation. Raw bytes preserved.
Delivery and retention (PayloadDownlinkApp, StorageManager)	Persist delivery status; retain progress only when the transfer protocol supports it. Missing confirmation keeps the product.	Ground confirms integrity; onboard validation matches product and attempt.
Configuration and limits (owning component, parameter service)	Retain approved configuration and version. Validate on load; use defaults or inhibit affected work if invalid.	Owning component checks ranges and compatibility before applying.

Data and owner	Persistence and reset behavior	Validity or confirmation
Boot and fault evidence (platform, MissionApp, SoHManager)	Capture reset cause on boot where supported. Retain bounded fault and event records.	Reports include boot identity and time validity. Missing evidence is not proof of a healthy reset.
Live health and context (subsystem managers)	Invalid or stale after reset until refreshed. Context already attached to a product stays with it.	Source managers report age and validity.
Queues and chunk buffers	Transient. Reinitialize after reset; cancel pending operations.	Queued work is not completed work.
Reduced-volume products (StorageManager)	Persist as separate entries with source IDs, reduction method and version, own integrity value. Partial if the source is partial or missing.	Complete after the reduction finishes and its integrity value is written. Confirmation does not release the source.
Uplink replay counter (critical parameter store)	Persistent. Triple-stored in FRAM with a CRC per copy, majority-vote read. Written before the command executes.	Command accepted only when its counter exceeds the last accepted value within the forward window. Current value reported in SOH.
SDLS keys and slot health (critical parameter store)	Persistent. Triple-stored in FRAM plus one read-only cold spare in NOR. Loaded on the EGSE bench; no in-flight rekey.	Slot health validated at boot and reported. Loss of the majority is a ground-recovery item.
Commissioning flags, deployment latch, last-contact time (MissionApp via critical parameter store)	Persistent. Flags change only by ground command; the deployment latch is set once on first Deploy exit; last-contact time updates on each authenticated contact.	Evaluated on every mode entry and by the contact timer. A missing record selects the least-capable state.

The command and telemetry dictionary is generated from the FPP model. Science metadata fields and external message formats are recorded in the data dictionary or ICD with a matching release version.

6.4. Ground system

Yamcs is the flight mission-control system. The F′ GDS, science receiver, and payload viewer remain the FlatSat tools. Ground integration must provide matching dictionaries, packet framing, command results, product reconstruction, integrity checks, and receipt acknowledgments. Flight COMMS and ground owners agree on framing, radio adapter, bandwidth, contact-interruption behavior, and command authorization. Receipt confirmation must tolerate contact loss and replay without corrupting product state.

6.5. Ground tools and workflow

EGSE scripts support bench checks and loading; the radio and ground-station layer transports frames; GDS or Yamcs handles commanding and telemetry; science tools validate, archive, and analyze products. Python is the starting point for test and analysis scripts. GNU Radio, Open MCT, and a database are candidates, not commitments.

Subsystem SOH downlinks without science products. During a pass, operators select the spacecraft and dictionary, check health, issue approved commands, verify results, receive and validate products, and record anomalies. Pass procedures belong in the MOP. Only one command authority is enabled for a spacecraft at a time, with explicit handover between GDS and Yamcs during integration. Bench power, deployment, and reset operations follow the applicable test procedure; software tools do not substitute for electrical interlocks.

7. Fault Detection, Isolation, and Recovery (FDIR)

This section defines how a fault is detected, which tier may act on it, and how the spacecraft recovers. FSW-level parents: FSW-007 (Safe entry on assigned indications), FSW-011 (rate monitor and inhibit), FSW-013 (safe boot), FSW-006 (48-hour SOH evidence). System-level parents: OBC-007, OBC-008, OBC-009, OPS-008, and OBC-005 (Section 7.3). FDIR is built on the F′ services in Section 3.2, the manager contracts in Section 4.2, and the CCSDS stack in Section 8; it does not depend on PUS services. [S2, S8, S11]

7.1. Safe entry and authority

Subsystem managers and framework health services report faults; MissionApp owns Safe entry and exit. FSW-007 separates the software transition from the sensor thresholds and hardware detection that produce the indication. The low-power threshold is allocated by OPS-011 (30 % state of charge) and owned by EPS and operations; FSW consumes the EPS indication and does not evaluate state of charge itself. Fault severity, persistence, thresholds, output states, and the power-positive Safe configuration require EPS, ADCS, systems, and operations approval (OD-03). Hardware protection is independent of software responsiveness. [S2]

7.2. Safe exit

Safe exit is ground-commanded. MOP v2.0 §8.8 defines the only exit as a manual transition, and FSW-001 binds the software to the MOP’s mode changes. When the command arrives, MissionApp enforces the exit guards: every required input is fresh and valid; every required input is within recovery limits and has stayed healthy for a configurable stability interval; no exit path bypasses separation inhibits or subsystem protections; a latched or nonrecoverable fault rejects the command until ground clears the latch. After a watchdog reset, the software first establishes the inhibited startup state FSW-013 requires and reassesses health; it does not restore

the interrupted activity. Automatic Safe exit is a proposal to mission operations (OD-03); if adopted, the same guards apply with a bounded re-entry policy. [S2]

The fault table records detection source, detection and clear conditions separately so hysteresis is possible, clear dwell time, whether a reset leaves a latch, whether automatic recovery is allowed, and the retry policy. Unknown or stale required health holds Safe. Faults classified as nonrecoverable or requiring inspection stay latched pending ground action. All limits and classifications are TBD.

Once the exit is commanded and the guards pass, MissionApp commands and confirms Base configuration, logs the transition and dispositions, then re-evaluates interrupted and queued science under Section 5.2. If the required state cannot be achieved or a critical fault recurs, the spacecraft stays in or returns to Safe. Table 7 lists the principal fault cases; an empty detection-source cell is a gap.

Table 7. Fault responses and detection sources

Condition	Detection source	Response and recovery evidence
Critical EPS or thermal indication	Device indication via EpsManager or ThermalManager (L1 into L2). Thresholds owned by EPS and thermal.	Stop or inhibit science; request protective states. Valid clear thresholds, hysteresis, and stability period before exit. Thresholds TBD (OD-03).
Payload timeout or reset	PayloadDriver transaction timeout (L1); PayloadManager achieved-state mismatch after reconciliation (L2).	Mark the attempt partial; reconcile payload state. Escalate to Safe only if the approved severity requires it.
ADCS angular rate exceeds the safety range	AdcsManager rate monitor on ADCS-reported rates (L2). FSW-011.	Command no control while exceeded; report the inhibit and cause. Enter Safe (MOP v2.0 §8.8 lists rate exceedance as a Safe anomaly). The monitor runs during Post-Deployment auto-detumble (MOP §8.2). The range and hysteresis are TBD by ADCS and systems and must not inhibit the ADCS-002 detumble.
Watchdog reset or repeated reboot	Hardware watchdog (L0), plus boot identity and reset cause read at startup (Section 9.3).	Capture reset cause; reconcile persistent state; boot to Safe (FSW-013). Retry latch depends on watchdog and boot design.
Storage full or corrupt product	StorageManager admission on reservation; per-product integrity on finalization and on reconciliation after reset (L2).	Reclaim confirmed products or block collection. Quarantine corrupt data and report.

Condition	Detection source	Response and recovery evidence
Contact or transfer interruption	CommsManager transport error or link-not-ready (L1); PayloadDownlinkApp session state (L2); MissionApp contact timer against persisted last-contact time (L4).	Retain product and delivery state; retry or resume per protocol. A single lost contact does not require Safe. No contact for 48 hours (TBC) enters Safe with the low-power beacon cadence (MOP v2.0 §§8.2, 8.8).
Stale required health or failed transition	Validity and staleness aging in the reporting manager (L2); MissionApp transition-confirmation timeout.	Treat achieved state as unknown; block affected actions; apply the configured severity. Recovery requires new valid observations.
Health ping timeout, queue overflow, assertion or FATAL	F′ Health, ComQueue or component queue overflow, assertion or FATAL event (L3).	Report to MissionApp for severity evaluation. A Health ping proves software responsiveness only.
Failed uplink authentication or replay rejection	N2 crypto component and command admission (L2); Section 8.7.	Count, event, rate-limit. Never enters Safe and never latches out commanding: noise on the uplink must not take the spacecraft out of contact. Rate-limit parameters TBD (OD-13).
Critical parameter store slot failure	Majority-vote read reports a CRC failure or disagreeing copy (Section 7.3).	Continue on the majority; rewrite the failed copy where writable; report slot health. Loss of the majority is a ground-recovery item (L5).
Code region CRC mismatch	Periodic CRC of the code region against the golden CRC in FRAM or NOR (Section 7.3).	Reset and reload the image from NOR. Scrub rate and reload path TBD (OD-14).

Table 8 records which tier may act on a fault. A tier below L4 that changes spacecraft mode is a design error.

Table 8. Fault tiers and recovery authority

Tier	Detects	Authority
L0 hardware	EPS latch-up and overcurrent trips; the hardware watchdog.	Acts independently of software and reports afterward. Software cannot suppress it.
L1 driver	Transaction timeout, framing or CRC error, register readback mismatch, out-of-range raw value.	Reports up with the failing transaction identity. Never changes mode.

Tier	Detects	Authority
L2 manager	Achieved-state mismatch, staleness and validity aging, illegal request for the current state, the FSW-011 rate monitor.	Reports up; may inhibit its own subsystem where the requirement says so. Never changes mode.
L3 component and framework	Health ping timeout, queue overflow, assertion, or FATAL.	Reports through framework health and event services. Never changes mode.
L4 mission (MissionApp)	Aggregated indications from L1–L3 against the approved fault classification.	The only tier with Safe entry authority and with exit-guard enforcement on a commanded exit.
L5 ground	Latched and nonrecoverable faults; faults requiring inspection.	Commands Safe exit, clears latches, authorizes actions the spacecraft may not take alone. Requires a contact.

One evidence path survives software failure. COMM-009 requires COMMS to store and transmit beacon packets independently of the flight software in contingency cases. That beacon originates in the COMMS board, so signing it is not a software responsibility (Section 8.3).

7.3. Memory, state integrity, and radiation

For upset symptoms such as corrupted data or state, the software validates ranges and integrity, rejects suspect data, restores a verified copy where one exists, and reports the affected product or service. F′ provides no mission-specific radiation protection. [S9]

No vendor document states whether iOBC memory is ECC-protected; the datasheet, brochure, and Quick Start Guide are all silent [S4, S12]. This design assumes no ECC. The consequence is that classic EDAC scrubbing is unavailable: with no syndrome to compare against, a read-and-rewrite pass over SDRAM detects nothing. Table 9 scopes each mechanism to a region where a reference copy exists. Confirming ECC status on the delivered unit is OD-14.

Table 9. Memory and state integrity by region

Region	Mechanism	Note
Code (.text)	Periodic CRC of the loaded code region against a golden CRC in FRAM or NOR. On mismatch, reset and reload from NOR.	Bounded work at a low rate against a reference copy. On the vendor path the application is stored in NOR above the bootloader and copied to SDRAM at boot [S4, S12]. A loader-to-RAM model would place the image on SD instead (OD-01). Scrub rate and CPU cost TBD (OD-14).

Region	Mechanism	Note
Keys, replay counter, critical configuration, mode latches, schedule state, commissioning flags	Triple-store in FRAM, a CRC per copy, majority vote on read, slot health reported as SOH.	One component, the critical parameter store, serves all of these. FRAM is byte-writable and non-volatile.
Science products, raw and reduced	Per-product integrity values and partial marking (Section 6).	No change.
SDRAM heap and stack	Not scrubbable in software. Mitigated structurally: initialization-time allocation (Section 12.3), stack canaries, the hardware watchdog, and a scheduled-reboot policy if approved.	Recorded as unmitigated by software so the residual lands with hardware and operations.
Device registers	Periodic driver readback against the last commanded configuration.	Catches silent upsets in peripheral configuration. Readback period per device TBD (OD-06).

The datasheet states that FRAM cells are not susceptible to single-event upsets in the way SRAM cells are, and that the memory controller remains susceptible [S4]. Critical state therefore lives in FRAM rather than SD or SDRAM, and three copies with a majority vote protect against cell-level corruption while sharing one controller. Slot-health reporting detects the controller case; surviving it is a ground action. The vendor keeps one SD card powered off while the other is in use for the same reason.

Critical parameter store. One FRAM-backed component provides the second row of Table 9 to every client. Each record is written to three copies with a CRC each; a read returns the majority value and reports any disagreeing copy; slot health is validated at boot and reported through SoHManager. One read-only cold spare of the key material is held in NOR. Clients: the SDLS key manager through the Svc.Ccsds.SdlsKeyInterface, the uplink replay counter (written before a command executes), and critical configuration, mode latches, schedule state, deployment latch, last-contact time, and commissioning flags. Record sizes, FRAM budget, and the policy when the vote itself fails are TBD (OD-14). [S4, S8]

A scheduled-reboot policy is an open question. Periodic reboot is common practice on non-ECC cubesats and costs little because the boot path already reconciles persistent state. It interacts with Section 5.2 restart semantics and in-progress downlinks, so it is a mission-operations decision (OD-14).

OBC-005 requires the OBC to detect and recover from memory corruption that could affect spacecraft operation or stored mission data, and its rationale accepts reboot or reset as a

mitigation. Without ECC, Table 9 is how the software meets it. Stored mission data: per-product integrity detects; retention and re-downlink recover. Critical state: per-copy CRC detects; majority vote and the NOR cold spare recover. Code: periodic CRC detects; reload from NOR recovers. Live SDRAM working memory: not detectable in software; recovered only by watchdog reset or scheduled reboot, which the rationale allows. That last case is the residual the OBC owner accepts or closes with hardware. OBC-005 is allocated to the OBC subsystem; OD-14 recommends an FSW-level requirement naming these mechanisms. [S2]

For latch-up and overcurrent, EPS and hardware owners define isolation or power cycling; a software reset alone is not an adequate response. Accumulated-dose degradation is a hardware qualification matter; the software trends health and responds to observable limits. Fault injection tests response logic, not radiation tolerance. Separation inhibits, deployable actuation, critical load switching, watchdog and reset behavior, Safe entry and exit, command admission, and science integrity are critical paths for review and fault injection.

8. Command Authentication and Data Protection

8.1. Requirement basis

UNP12-92 states that uplink communications shall be encrypted for all spacecraft. It appears on the Ground System sheet (Inspection, Complete) and the COMMs sheet (Test, Not Complete) and is not allocated to FSW. Onboard decryption and verification must happen in the flight software or in the COMMS board firmware. This section is the design for the flight-software case; the allocation is OD-13. The Ground System sheet’s Complete status cannot stand until the spacecraft side exists; that correction is also in OD-13. [S2]

UNP12-93 requires the downlink encryption approach to be discussed with UNP. It requires a recorded discussion, not downlink confidentiality. An authentication-only downlink satisfies it once discussed. This section is the artifact for that discussion (OD-13).

The initial protection profile in Table 10 is the restrictive one because it is valid under every licensing outcome, so the protection design is not dependent on the license schedule. Part 5 experimental licensing is being pursued. MOP v2.0 §8.2 still cites amateur beacon rules; reconciling the licensing basis is a PM and systems item, and the design does not depend on it.

8.2. Threat scope

In scope: unauthorized commanding; replay of a captured valid command; accidental commanding of the wrong vehicle, kept because two spacecraft sharing a ground system was planned; and compromise of a ground credential, where the question is what the spacecraft still enforces alone. Out of scope: science confidentiality in orbit, pending the UNP12-93 discussion; and ground-segment information security, which GS-003 assigns to the ground system. [S2]

8.3. Protection policy by data class

Policy is applied per data class through the SDLS security association (SA) index, not the virtual channel: the F′ v4.3.0 TmFramer supports one virtual channel per instance, so the SA index is the available policy knob (Section 8.5). The SA index is a 16-bit field in the frame context, and the router maps it to a crypto instance. [S8]

Table 10. Initial protection policy by link and data class

Link or data class	Initial SA policy	Note
UHF command uplink	Encrypt and authenticate	Required by UNP12-92.
UHF beacon and state of health	Authenticate, clear	Supports FSW-006. Health data operators need during a contingency must be readable.
UHF reduced-volume science	Authenticate; encrypt-capable	FSW-012 backup path. Encryption by configuration if UNP12-93 calls for it.
S-band science products	Authenticate; encrypt-capable	FSW-008 primary path. Same configurability.
Contingency beacon	Not a software path	COMM-009: COMMS produces this beacon independently of the software.
Host and FlatSat deployments	ClearText components or test keys	Flight keys never exist on a student machine or on FlatSat.

The protection stage is configurable per SA index. Moving to encrypt-both after licensing and the UNP12-93 discussion is a change to the SA map and key set, not to components, ports, or topology. Cipher and MAC selection is OD-13.

8.4. Design in F′ v4.3.0

F′ v4.3.0 ships the CCSDS stack and the SDLS integration interfaces. The standards implemented are Space Packet 133.0-B-2, TM 132.0-B-3, TC 232.0-B-4, and AOS 732.0-B-5. FSW-009 requires CCSDS telemetry Space Packets, with edition/profile confirmation still open; the mission packet profile (APIDs, virtual channel, SA indices) is OD-13. Protection is applied at the transfer-frame boundary, so Space Packet structure and FSW-009 conformance survive it. [S2, S8]

The shipped default is a no-op: ClearTextEncryptor and ClearTextDecryptor pass frames through, and their model files say so. N2 supplies one component implementing the CcsdsSdlsEncrypt and CcsdsSdlsDecrypt interfaces, selected through the ComCcsdsSdlsConfig module. This is topology wiring and component substitution; no framework source is modified. No SDLS compliance is claimed: the framework implements the SA-indexed protection pattern

and cites no CCSDS 355.0-B publication. Verifying the N2 component against that standard is separate work. Appendix A lists the components, ports, and interfaces involved. [S8]

8.5. Three constraints

First, the crypto component must be passive with synchronous handlers. SdlsSaRouter requirement SVC-CCSDS-SDLS-SA-ROUTER-008 requires downstream crypto components to invoke the return ports synchronously on the caller's thread; an asynchronous or hardware-backed component cannot be connected without external synchronization. On a 400 MHz ARM9 without cryptographic acceleration, the cipher therefore runs on the framing thread. Its cost is a Section 10 gate measurement, and under a cooperative scheduler a synchronous cipher call blocks every other task (Section 9.1). [S4, S8]

Second, per-channel separation is by SA index. TmFramer supports exactly one virtual channel per instance (SVC-Ccsds-TM-FRAMER-008); TcDeframer accepts all virtual channels or one configured channel, so the constraint is on the downlink side. Multi-channel framing would need multiple framer instances with no pre-built pattern. [S8]

Third, bulk science shares the framer and SDLS stage with telemetry and events. FileDownlink feeds ComQueue's FILE lane with its own depth and priority, and BufferManager has a separate larger bin for file buffers, but everything downstream of ComQueue is shared. Science gets a different SA index, not a different path. [S8]

8.6. Frame profile

AosFramer and AosDeframer do not support SDLS, the Insert Zone, or the Frame Header Error Control Field. AOS is otherwise the more flexible choice for virtual channel management; choosing it forfeits the SDLS processing stage. The FSW-009 frame profile and the protection design are therefore one decision, OD-13, owned by COMMS with the FSW lead. COMM-008 permits AX.25, which would conflict with FSW-009 and SYS-009, and its amateur-ground-station rationale predates the Part 5 plan; the COMMs requirement owner and systems engineering resolve it (OD-13). [S2, S8]

8.7. Key handling and replay protection

Keys are pre-loaded symmetric keys with no in-flight rekey. Rekey is the largest single source of lockout risk: an interrupted, misordered, or misapplied rekey leaves a spacecraft that cannot be commanded. If a flight key is compromised, the response is operational: stop commanding until the situation is understood.

Keys live in the critical parameter store (Section 7.3) with one cold spare in NOR. F′'s SdlsFileKeyManager reads a key from a file per request and is not used; N2 substitutes its own key manager behind the Svc.Ccsds.SdlsKeyInterface. [S8]

Replay protection is a monotonic counter in FRAM. A command is accepted only when its counter exceeds the last accepted value within a forward window wide enough to tolerate uplinks lost to a bad pass. The counter is persisted before the command executes; persisting afterward leaves a reset window for replay. Window width is TBD (OD-13). Resynchronization: the current accepted counter is downlinked in SOH; operations send the next command with counter plus one. The counter is not secret.

There is no unauthenticated bypass and no recovery backdoor. Recovery from a damaged key slot is via the redundant slots and the NOR cold spare. Failed verification is counted, evented, and rate-limited; it never enters Safe and never latches out commanding, because otherwise anyone able to transmit on the uplink frequency could take the spacecraft out of contact.

Key loading is an EGSE bench procedure with a verification step before the unit is closed out. FlatSat and host deployments use ClearText components or test keys. Flight keys never enter the repository. Export-control review of any cryptographic source in the repository is an OD-13 item owned by the principal investigator.

8.8. Requirement trace and non-goals

UNP12-92 is the parent for uplink encryption. FSW-010 requires rejection of ground or internal commands that fail command-format checks when received or before execution; it does not establish authentication or replay checks. An explicit requirement for command authentication and replay rejection is recommended (OD-13). Non-goals: in-flight rekey, an unauthenticated bypass, science confidentiality pending UNP12-93, signing the COMM-009 beacon, CCSDS 355.0-B compliance, and ground-segment security. [S2]

9. Flight Platform and Configuration

9.1. iOBC feasibility milestone

The controlled datasheet specifies a 400 MHz 32-bit ARM9 (AT91SAM9G20), 32 MB SDRAM, 1 MB NOR code storage, 256 kB FRAM, dual 2 GB SD cards, and an SDK built on FreeRTOS with an ARM GCC toolchain, an ISIS hardware abstraction layer, and a FAT32 filesystem [S4]. The product brochure states 64 MB SDRAM; this document follows the datasheet. Both are advertised capabilities; the delivered board is confirmed under OD-01. ARM9 is not the ARMv9 architecture.

In F′ v4.3.0, the only FreeRTOS entry in the supported-platforms table is a community reference project for an Adafruit Feather M4 with no continuous-integration status; the FreeRTOS OSAL backend is a community package, and bare-metal ARM boards appear only under “Targeted Platforms (Planned Support)”. The iOBC is not a supported platform. The milestone is therefore a port that the team owns and maintains. No iOBC has been procured, no target build exists, and every performance statement in Sections 4, 8, and 9 is unmeasured. [S8, S13]

Application code budget. On the vendor path, the application image is stored in NOR above the bootloader (0x0 to 0xA000) and copied to SDRAM at boot [S4, S12]. The binding constraint is the 1 MB NOR, not the SDRAM. The internal studies measured F′ binaries on an ARMv6 Linux proxy, not this architecture or OS, so they bound the real figure loosely [S13]. A target-correct build settles it. If the image does not fit, the options are a loader-to-RAM model with the image on SD, an older framework release (which removes the SDLS integration interfaces Section 8 uses, coupling this to OD-13), a reduced topology, or a different flight computer (OD-01).

Scheduling model. The vendor states that FreeRTOS on this board is configured without pre-emptive task switching; a task switch occurs when a task yields [S4]. F′ active components assume pre-emptive, priority-scheduled threads, and Sections 4.3, 4.4, and 8.5 depend on that. Whether the configuration can be changed is a port question (OD-01). Fallback if it cannot: build the flight topology from passive and queued components only, dispatched by one rate group on a single task; bound every handler to a measured worst-case time; run payload, storage, and radio transfers as incremental state machines with explicit yield points; and kick the watchdog from the dispatcher so a handler overrun trips the hardware watchdog instead of stalling silently. This gives up the priority separation Section 4.3 assumes and is accepted only with measured worst-case latencies for command, health, and fault response.

OSAL port. A FreeRTOS backend exists as a community package for a different board. Establishing and maintaining one for the iOBC is project work with a named owner; the FreeRTOS caveats in the framework's integration guide have not been reviewed against this design (OD-01). [S8]

Once hardware and vendor artifacts are in hand, the first milestone is a reproducible cross-build and boot of a minimal F′ deployment that accepts a command, emits telemetry and events, exercises task, queue, and time services, records reset behavior, and reports the stripped image size against the 1 MB NOR budget. CfdpManager carries an ATTRIBUTION.md marking imported source; CFDP is not in this design, and adopting it later requires a license review. [S4, S8, S12]

9.2. Resource and release control

We measure executable placement, RAM, stack peaks, queue and buffer occupancy, CPU utilization, storage capacity, write rate, and downlink backlog on each target. The flight budget includes OS and SDK overhead, storage-copy policy, partial products, metadata, and worst-case contact gaps. Table 14 is the current budget; most rows are unmeasured. Four measurements are required: stripped image size, SDLS cipher cost on the framing thread, code-region CRC scrub cost, and science-reduction cost.

Table 14. Software resource budget

Item	Allocation or current figure	Owner and evidence
NOR application image	1 MB total; bootloader 0x0–0xA000 (40 kB); about 984 kB available. Stripped image size TBD.	FSW. Target-correct build (OD-01).
SDRAM working memory	32 MB on board. No program allocation exists; System Budgets v2.1 carries no SDRAM line [S14].	FSW / systems. Measured peak on target.
FRAM critical parameter store	256 kB on board. Record sizes TBD.	FSW. OD-14.
State-of-health storage	Received SOH records retained for 48 h, about 978 kB (FSW-006, OBC-002, System Budgets v2.1).	FSW / OBC. Measured write rate.
Science product storage	13 Mb per orbit downlink (COMM-001). Product size, cadence, retention TBD.	FSW / payload / systems. OD-02.
Usable mass storage	2 GB, one SD card active at a time [S4]. System Budgets v2.1 sums both cards; if the redundant configuration flies, Section 6.2 is sized against the wrong number.	Systems / OBC. OD-01.
Uplink	At least 50 kbps (COMM-002). Command rate and packet sizes TBD.	COMMS / FSW. Measured.
Downlink throughput with SDLS	TBD. Science and telemetry share the framer and cipher (Section 8.5).	FSW. Table 11 gate.
CPU	400 MHz ARM9. Cipher, code-CRC scrub, and science-reduction costs unmeasured.	FSW. Table 11 gate.
Stacks, queues, Health timeouts	TBD per deployment.	FSW. Target tests.

The neutron_2 engineering model remains the comparison baseline. It builds against F′ v4.2.1; the design targets v4.3.0, and the upgrade is OD-01 work. Each release pins the framework, toolchain, generated dictionaries, configuration tables, ICD versions, and driver implementations in a release manifest.

9.3. Boot and update boundary

At startup, the software records boot identity and reset cause, validates persistent configuration through the critical parameter store, reconciles schedules and product state, and applies inhibits

before enabling mission activity (FSW-013). Unknown persistence is never read as proof that deployment actions or transfers completed. EPS-009 defines the power-on order (EPS, then OBC, then COMMS); the software assumes only that the OBC has power at boot and treats every other subsystem as unpowered until its manager confirms it. [S2]

The proposed first-power-on rule enters Deploy Mode when no deployment record exists and starts the MOP v2.0 §8.1 30-minute (TBC) timer. FSW-013 requires a mission-operations-defined safe mode with minimal subsystem activation after reset or power cycle and states no deployment-latch exception; mission operations and systems engineering must reconcile the proposed first-boot rule with that requirement (OD-07). The timer value, deployment latch, last-contact time, and commissioning flags are critical-parameter-store records. Mission operations and systems engineering approve this rule (OD-07).

In-flight software update and rollback are not established. Before enabling updates, the team must prove image integrity and compatibility checks and a vendor-supported recovery path that works when the candidate image cannot run (OD-09).

10. Development and Verification

Table 11 is the roadmap from concept to flight acceptance. Existing demonstrations are regression references; each new driver is tested through its shared manager contract. Source inspection, host test, FlatSat run, and flight-target test are different levels of evidence and every result records which one it is.

Table 11. Development and verification gates

Stage	Exit evidence
Concept and shared design	Component ownership, mode and transition assumptions, science format, recovery policy, protection policy, and interface contracts reviewed. Open items linked to Table 13.
Artemis regression	The existing FlatSat demonstration (commanding, simulated collection, storage, ground return) reproduces its recorded results against the current development snapshot. Build tag, configuration, hardware or simulator identity, and as-run evidence are recorded. Closes no flight-target requirement.
Real payload FlatSat	Simulator adapter replaced by the real payload driver. Collection, return to Standby, transfer, storage, and ground receipt demonstrated, including timeout, abort, and the required collection duration.
Optional Artemis PDU increment	Verified board, protocol, and power arrangement; demonstrated command, readback, and failure reporting. Kept distinct from flight EPS bring-up.

Stage	Exit evidence
iOBC platform feasibility	Minimal F′ build and boot on the iOBC; command and event path; OSAL and platform behavior; reset evidence; measured resources including image size, SDLS cipher cost, and code-CRC scrub cost. OS and port decided from results.
Flight subsystem integration	Prototype drivers replaced under approved ICDs; tested nominal and malformed, timeout, and reset cases; manager contracts stable.
Integrated flight-target operations	Collection, storage, and downlink under load; Safe entry and commanded exit; interrupted-science restart; storage reclamation; contact loss and the 48-hour timer; reboot; day-in-the-life; authenticated commanding end-to-end with replay rejection and counter resynchronization; simultaneous science downlink and telemetry.
Flight acceptance	Applicable RVM tests closed on identified flight hardware and configuration; results, anomalies, release manifest, and approved operational limits archived. FlatSat results do not close target-dependent requirements.

10.1. Testbeds and evidence

The student implementing a component maintains its unit tests and simulator fixtures. The software lead assigns a FlatSat integration owner; subsystem leads provide representative devices, ICD test vectors, and constraints. Host tests run for each change, the shared demo for integration changes, and target or HIL tests before any hardware claim. Each log records build tag, configuration, hardware or simulator identity, procedure revision, expected and actual result, and anomaly disposition.

Deterministic host scenarios cover nominal collection, invalid commands, stale health, timeouts, contact loss, storage pressure, and recovery. The FlatSat setup guide records location, wiring and power, reservation owner, startup and shutdown steps, and result storage. The UNP integrated Five Tests (simulated communications, complete charge cycle, command execution, ADCS verification, day in the life) are retained in the project test plan with software evidence linked to the current UNP procedure, which systems engineering confirms (OD-12).

10.2. Requirement allocation

Table 12 allocates the fifteen requirements on sheet 6.7 FSW. RVM wording is authoritative; the evidence column is proposed acceptance evidence, not a closure claim. The RVM assigns Test to FSW-001–012 and FSW-014–015, and Demo to FSW-013; all are Not Complete. Additional design checks below do not broaden the requirement wording. The required FSW-014 service set remains to be confirmed (OD-16). [S2]

Table 12. Requirement allocation

RVM ID	Owner and section	Proposed evidence
FSW-001 Mode transitions	MissionApp; §5, §7	Every permitted transition from every trigger type reaches its destination with entry conditions met; rejected and held transitions recorded.
FSW-002 Real-time command execution	Command service; §3.2, §4.2, §8	Each valid command executes on release; release, validity, and action are separately evidenced.
FSW-003 Time-based scheduling	MissionApp; §4.3, §5	One execution per loaded time trigger; expiry, invalid input, and revalidation across reset.
FSW-004 Location-based scheduling	MissionApp, GpsManager; §4.3, §5	One execution per loaded location trigger; triggers with an invalid position source are held.
FSW-005 Science acquisition context	ScienceApp; §6.1	Recorded time, latitude, longitude, altitude, attitude, and magnetic-field direction and magnitude match known injected values for the measurement interval. Product association and validity flags implement the design.
FSW-006 SOH availability (48 h)	SoHManager, StorageManager; §6.2, §7	Each SOH record received from a powered subsystem for the previous 48 hours is stored and available to COMMS for downlink.
FSW-007 Autonomous Safe entry	MissionApp; §7.1, §7.2	Each low-power, watchdog-timeout, or other indication assigned to trigger Safe Mode causes entry. ADCS-rate and contact-timer cases follow the approved MOP fault policy; TBC thresholds remain open.
FSW-008 Selected data transfer	PayloadDownlinkApp, CommsManager; §6.2, §8.3	Selected products and telemetry arrive complete at the COMMS input on a valid downlink command.
FSW-009 CCSDS packet formatting	Framing chain; §8.4	All mission data sent to COMMS for ground transmission meets the CCSDS telemetry Space Packet standard. CCSDS 133.0-B-2 EC2 §4.1 and the mission profile remain proposed; verify structure after SDLS protection.
FSW-010 Invalid command rejection	Command-format validation; §4.2, §8.8	Ground and internal commands failing specified format checks are rejected when received or before execution. The check specification remains to be identified (OD-16); authentication and replay are separate design checks (OD-13).

RVM ID	Owner and section	Proposed evidence
FSW-011 ADCS rate-limit response	AdcsManager; §3.4, §7.2	Monitored angular rates exceeding the accepted safety range cause a No Control command. Inhibit reporting, Safe entry, and clearing conditions are additional design/MOP checks.
FSW-012 Reduced-volume science	ScienceApp, StorageManager; §5, §6	Selected science is compressed or summarized into a product suitable for UHF downlink when S-band is unavailable. The design keeps a separate catalog entry naming its unchanged source.
FSW-013 Safe mode at boot	MissionApp, platform; §7.2, §9.3	Demonstrate boot to the mission-operations-defined safe mode with minimal subsystem activation after reset and power cycle.
FSW-014 Operational services	Framework services, applications; §3.2, §4.2	Test each confirmed required CTL service against its row. Request Verification is in rows 2–7; other operational services are in rows 8–84 and 98–100. Required scope remains open (OD-16).
FSW-015 Subsystem command and telemetry	Managers and drivers; §3.4, §4.1	Test raw-command forwarding and response return for EPS, ADCS, OBC, COMMS, and Payload services in CTL rows 85–97. Timeout reporting as unknown state is an additional design check.

Three behaviors the software implements have no FSW-level requirement: the Deploy-mode inhibits (UNP12-96, allocated to the satellite system), receiving the payload-to-OBC transfer (PAY-004 covers only the payload side), and the Application–Manager–Driver portability that Sections 3 and 4 and Table 11 depend on. FSW-015 covers raw Payload command forwarding and responses; any additional mission-specific configuration workflow needs explicit allocation. The design is kept; the gaps are OD-15. The FSW source references have been corrected; the remaining service-scope, SOH acceptance-detail, command-format, and packet-profile questions are tracked in OD-16. [S2]

11. Open Decisions

Owners below are functional reviewers; individual assignees and dates are not yet committed. Each item closes before its dependent gate in Table 11.

Table 13. Open decisions

Item	Owner and closure
OD-01 iOBC platform	OBC / FSW / systems. Confirm the delivered board configuration (SDRAM, ECC). Build a target-correct minimal deployment and report stripped image size against 1 MB NOR. Select framework version, runtime model (NOR-resident image or loader-to-RAM), and scheduling configuration. Name the OSAL port owner. Closes before flight subsystem integration.
OD-02 Science data budget	Systems / payload / operations. Confirm CSV, product size, cadence, windows, metadata, storage and contact budgets. Approve the FSW-012 reduction algorithm, its CPU and memory cost, and derived-product integrity.
OD-03 Safe configuration	Systems / EPS / ADCS / operations. Approve protective output states, power balance, the fault list including ADCS rate exceedance and 48-hour no-contact, clear thresholds and dwell, and the exit guards enforced on a commanded exit. Automatic exit requires a MOP change first.
OD-04 Restart semantics	Payload / operations. Approve restart with a new attempt ID, the remaining-window rule, partial-product handling, and retry limits.
OD-05 Retention and receipt	Systems / ground / FSW. Approve oldest-confirmed reclamation, integrity acknowledgment, reserve, protected products, and log allocation.
OD-06 Interface closure	Subsystem / OBC owners. Freeze driver ICDs, bus selection, timeouts, data formats, achieved-state checks, and register readback periods.
OD-07 Flight modes and inhibits	Systems / operations. Approve the seven-mode matrix, guards, separation timer handling, and Post-Deployment exit authority. Reconcile proposed Deploy entry on first power-on with FSW-013 safe boot after reset or power cycle; no deployment-latch exception is stated in the requirement.
OD-08 Time and context	OBC / ADCS / GPS / payload. Confirm time and location sources, the OBC-010 accuracy budget, drift measurement, the ground time-update command, and record association.
OD-09 Ground and updates	Ground / COMMS / FSW. Verify the Yamcs adapter and receipt path. OBC / operations. Decide whether in-flight update is required and demonstrate recovery before enabling it.
OD-10 Configuration ownership	FSW / systems. Reconcile remaining MDD and MOP two-spacecraft assumptions with the single-satellite design. All fifteen FSW requirements are marked “N” in “Differs for 2 s/c?”; spacecraft identifiers remain a design choice.
OD-11 Models and feedback	Software / systems / PM. Assign the diagram and model owner; decide whether Capella is authoritative.

Item	Owner and closure
OD-12 Assurance and test resources	Software / systems / operators. Confirm assurance obligations, coverage criteria, testbed roster, and the current Five Tests procedure.
OD-13 Command authentication and packet profile	Systems / COMMS / FSW / PI. Allocate UNP12-92's onboard half to FSW or COMMS firmware and correct the Ground System sheet status. Hold and record the UNP12-93 discussion. Approve APIDs, virtual channel, SA indices, and TM versus AOS (AOS forfeits SDLS). Reconcile COMM-008. Select cipher, MAC, and replay window. Add an FSW requirement for authentication and replay rejection. PI: export-control review. Closes before flight subsystem integration.
OD-14 Memory and state integrity	OBC / FSW / systems. Confirm ECC status of the delivered iOBC. Allocate code-CRC scrub rate, critical parameter store sizes and FRAM budget, and the failed-vote policy. Decide scheduled reboot. Add an FSW requirement naming the OBC-005 mechanisms. Closes before iOBC feasibility.
OD-15 FSW allocation gaps	FSW lead / systems. Allocate or accept in writing: Deploy inhibits (UNP12-96), payload-to-OBC receive (PAY-004), portability layering, and any mission-specific payload configuration workflow beyond FSW-015 raw-command forwarding.
OD-16 RVM traces and data quality	FSW and satellite-system requirements owners. Confirm the required CTL service set for FSW-014; decide whether active-mode information is an additional FSW-006 acceptance condition; identify the command-format specification for FSW-010; confirm the proposed CCSDS edition and mission profile for FSW-009 with OD-13. Source references and verification methods/statuses are populated; these remaining questions close before verification closure.

The software lead maintains this document with subsystem and mission-operations contributors. Changed assumptions are reviewed before the dependent gate; decisions link to the issue tracker; design, ICD, and tests update together. Unresolved items stay until their owner supplies evidence or approves the allocation.

12. Student Development and Team Workflow

12.1. Weekly work and definition of done

The team runs a weekly Scrum: prioritize the GitHub backlog with subsystem dependencies, plan a small testable goal, check blockers mid-week, demonstrate at the review, record one improvement. Each issue names the owner, reviewer, acceptance behavior, affected interface or requirement, and evidence location.

A task is done when the change meets its acceptance criteria, passes build and tests, has an approved review, and includes updated interface or design notes and reproducible evidence. If hardware is unavailable, only the scoped simulation or source task closes, with a linked hardware-verification issue. An incomplete target test is never relabeled as a successful demo.

12.2. Branches and review authority

Students branch from develop and open pull requests back into develop with the issue, behavior change, test evidence, and any ICD or dictionary change. A lead or designated reviewer approves. Develop stays buildable with its demonstrated configuration identified.

Only leads or senior developers manage pull requests into release, release tags, and hotfixes. A reviewed develop snapshot is promoted after integration checks, tagged at the exact tested commit, and archived with its configuration and results. A tag identifies evidence; it does not certify flight readiness. Hotfixes branch from the affected release, go through review and test, merge to release with a new tag, and are carried back to develop by a reviewed pull request.

12.3. Coding and verification practices

Model ports, commands, events, telemetry, parameters, and bounded types in FPP before writing C++ handlers. Generated code and dictionaries follow the model and are never hand-edited. Comments state intent, units, preconditions, ownership, and non-obvious fault behavior. [S8]

Flight paths use bounded buffers and initialization-time allocation and validate external lengths, ranges, states, and identifiers. Assertions are for internal invariants, not malformed ground or device input. The toolchain's C++ standard, exception, and RTTI rules apply, and vendor-library compatibility is proven on the iOBC. Power-of-Ten guidance is a review aid with documented exceptions.

Before review: unit tests, topology and integration checks, compiler warnings, formatting, and available static analysis, covering nominal, boundary, timeout, invalid-input, and recovery paths. Coverage gaps are recorded; the target percentage requires team allocation. Code touching the crypto component, key store, or replay counter is a critical path and requires lead review. Flight keys never enter the repository.

12.4. Onboarding

A new student builds and runs the pinned host or FlatSat example, traces one command to its handler and one telemetry value to ground, makes a small tested change, and has another student reproduce it. Learn C++, FPP, Git and pull requests, the GDS, and the relevant test procedure before owning a critical path.

The pinned F′ v4.3.0 checkout (commit 7d8f579f159d2f7c2d4984d92828575e37f87fa6) is the reference for ports, components, topologies, the Application–Manager–Driver, rate-group, manager-worker, and health-checking patterns, unit testing, and porting. fprime-course-materials

provides guided exercises; fprime-yamcs (reviewed at 5aafc72996c0cb1d290ceba11d46cf826cc831ec) covers ground integration. Onboarding instructions live with the repository. [S8]

Appendix A. How Commands and Telemetry Move Through F′

Section 8 refers to pieces of the F′ v4.3.0 communications stack by name. This appendix walks through what those pieces do so the design can be read without the framework source open. Everything here was checked against the pinned checkout [S8]. Component names appear once each so a student can find them in the repository.

The path down: telemetry, events, and science to the radio

Every downlinked item starts as a buffer handed to a queue. The queue is the first place the design gets any control: it has separate lanes for telemetry, events, and files, each with its own depth and priority, so a large science file waiting to go out cannot starve a health packet. The file lane draws from a separate, larger pool of buffers than the packet lanes, which keeps science transfers from exhausting the small buffers telemetry needs.

From the queue, each buffer is wrapped in a CCSDS Space Packet. The packet framer stamps it with an application process identifier, the APID, which tells the ground what kind of data it is, and with a sequence count that increments per APID so gaps are detectable. A small bookkeeping component keeps those counts. This is the layer FSW-009 cares about: after this step, the data is a standard Space Packet and stays one all the way to the ground.

Packets are then gathered into a transfer frame. The TM framer builds the frame, tags it with a virtual channel identifier, maintains its own frame counters, and appends a 16-bit cyclic redundancy check so the ground rejects a corrupted frame rather than decoding it into garbage. In this framework, the TM framer handles exactly one virtual channel per instance. That single fact drives Section 8.3: because the design cannot cheaply give science and telemetry different virtual channels, it separates them with the security association index instead.

Finally, the frame goes to a stub that hands bytes to whichever driver talks to the radio. The stub exists so the same stack works over a UART on FlatSat and over the SPI link to the flight radio.

The path up: a command from the radio to its handler

Bytes arriving from the radio go to the same stub, then to an accumulator that watches the stream for the start of a valid telecommand frame. The accumulator uses a detector tuned to the CCSDS telecommand frame format; until it sees a well-formed header and length it discards noise. This is the first defense against a bad pass: garbage never reaches the deframer.

The telecommand deframer strips the frame and checks it. Unlike the TM framer, it can accept all virtual channels or be pinned to one, so on the uplink side the virtual channel could be used as

a filter if the packet profile calls for it. The Space Packet deframer then recovers the packet and reads its APID, and a router sends the packet to the command dispatcher or, for uploads, to the file uplink service. The command dispatcher looks up the opcode, validates arguments, and calls the handler in the owning component. Mode permission, readiness, and inhibit checks (Section 4.2) happen in that handler, not in the stack.

Where protection sits

The SDLS processing stage uses a pair of components inserted between the Space Packet layer and the transfer-frame layer, one for each direction. On the way down, after packets are built but before the frame is sealed, a protection framer passes the data to a router that reads the security association index from the frame context and picks the crypto instance registered for that index. The crypto instance encrypts and authenticates, or only authenticates, according to that association, and the result goes on to the TM framer. On the way up the order reverses: the telecommand deframer hands the frame to a protection deframer, the router picks the instance for the index carried in the frame, and only a frame that verifies reaches the packet deframer. Because protection wraps the packet rather than altering it, the Space Packet the ground sees is the same one FSW built, and FSW-009 still holds.

The framework ships this SDLS processing stage with a pass-through crypto component whose own model file says it provides no confidentiality, integrity, or authentication. It is there so the stack can be tested in the clear. The N2 work is one component that implements the same encrypt and decrypt interfaces with a real cipher and MAC, plus a configuration file that names it instead of the pass-through. No framework file is edited, which is why upgrades stay cheap.

The crypto component needs keys. The framework's shipped key manager opens a file and reads the key every time it is asked, which suits a Linux host and not a spacecraft with keys in FRAM. N2 replaces it with a key manager that implements the same key interface but reads from the critical parameter store of Section 7.3.

Two constraints that shape the design

The router calls the crypto component synchronously and expects it to answer on the same thread before returning. The router's own requirements document says so, and the reason is that the router holds a lock while it waits; a crypto component that queued the work and answered later would deadlock it. The consequence for N2 is that the cipher runs inside the framing thread on the ARM9 with no hardware acceleration, so every frame pays the cipher cost in line. That is why Section 10 requires the cost to be measured before the throughput budget is set, and why the cooperative-scheduler question in Section 9.1 matters: on that scheduler the cipher would stall every other task while it runs.

The alternative AOS frame format is more flexible for virtual channels, but the framework's AOS framer and deframer do not support the SDLS processing stage; their design notes list it as disabled. Choosing AOS for FSW-009 would therefore remove command authentication. The frame profile and the protection design are one decision, OD-13.

Names for the repository

Where	Names in the F′ v4.3.0 repository
Subtopologies	Svc/Subtopologies/ComCcsds (the stack); Svc/Subtopologies/ComCcsdsSdls (the SDLS subtopology)
Downlink, in order	Svc::ComQueue; Svc::Ccsds::SpacePacketFramer with Svc::Ccsds::ApidManager; Svc::ComAggregator; Svc::Ccsds::TmFramer; Svc::ComStub
Uplink, in order	Svc::ComStub; Svc::FrameAccumulator with Svc::FrameDetectors::CcsdsTcFrameDetector; Svc::Ccsds::TcDeframer; Svc::Ccsds::SpacePacketDeframer; Svc::FprimeRouter
SDLS processing stage	Svc::Ccsds::CcsdsSdlsFramer; Svc::Ccsds::SdlsSaRouter; Svc::Ccsds::CcsdsSdlsDeframer. Defaults Svc::Ccsds::ClearTextEncryptor and ClearTextDecryptor, selected in ComCcsdsSdlsConfig
Interfaces N2 implements	Svc.Ccsds.CcsdsSdlsEncrypt; Svc.Ccsds.CcsdsSdlsDecrypt; Svc.Ccsds.SdlsKeyInterface
Frame context	ComCfg::FrameContext: virtual channel as an 8-bit field, security association index as a 16-bit field
Requirements cited	SVC-CCSDS-SDLS-SA-ROUTER-008 (synchronous crypto); SVC-Ccsds-TM-FRAMER-008 (one virtual channel); SVC-CCSDS-TC-DEFRAMER-004 (one or all virtual channels)

Radiation references for Section 7.3 [S9]: NASA JSC, Radiation, https://www.nasa.gov/reference/jsc-radiation/; NASA JPL, ASIC Design Guidelines Section 3.4, https://parts.jpl.nasa.gov/asic/Sect.3.4.html. Accessed 14 September 2026. The v3 review-comment dispositions are in the file “SDD v3 Review Comment Dispositions” in the SDD folder.